Sree Narayana Guru College of
Engineering & Technology
Chalakkode P.O., Korom, Payyannur, Kannur-670307

MINI PROJECT FINAL PRESENTATION

Guided By:

By,
Group Members:

Sree Narayana Guru College of Engineering & Technology
Chalakkode P.O., Korom, Payyannur, Kannur-670307

ABSTRACT

• A web application that analyzes blood test reports using OCR technology.
• Extracts 100+ medical parameters and evaluates against standard health ranges.
• Provides personalized diet plans and fitness recommendations based on results.
• Built with React.js, Tesseract.js, and a Python/Flask ML backend.

INTRODUCTION

- Blood reports contain vital health information that most people struggle to interpret.
- This system automates blood report analysis and gives actionable health advice.
- Users simply upload a blood report image or PDF to get instant results.
- The app bridges the gap between medical data and everyday health decisions.

PROBLEM STATEMENT

- Manual interpretation of blood reports is time-consuming and error-prone.
- Most patients lack medical knowledge to understand lab report values.
- Personalized diet and fitness advice based on blood data is not easily accessible.
- Existing apps require costly subscriptions or doctor visits for basic insights.

EXISTING SYSTEM

- Existing systems rely on manual data entry by the user.
- No automated OCR-based extraction from blood report images.
- Generic health advice not tailored to individual blood report values.
- Most tools are expensive or require professional medical supervision.

PROPOSED SYSTEM

- Automated OCR scanning using Tesseract.js extracts values directly from images.
- Intelligent rule-based analysis evaluates 100+ blood parameters instantly.
- Personalized diet, fitness, and lifestyle recommendations for each user.
- Offline-first PWA with Firebase sync, works without a doctor or subscription.

LITERATURE SURVEY

Paper Title / Authors	Methodology	Advantages	Disadvantages
Automated Blood Report Analysis – Kumar et al., 2021	CNN-based image classification	High accuracy on scanned reports	Requires large labeled dataset
OCR for Medical Documents – Singh et al., 2020	Tesseract OCR + regex parsing	Open-source and cost-free	Low accuracy on handwritten text
Health Recommender Systems – Patel et al., 2022	Rule-based medical knowledge base	Fast and interpretable results	Cannot learn from new data
ML Disease Prediction – Ali et al., 2019	Random Forest on blood parameters	High prediction accuracy (91%)	Needs patient history data

REQUIREMENT ANALYSIS

Hardware Requirements

- Processor: Intel Core i3 or equivalent (minimum)
- RAM: 4 GB minimum, 8 GB recommended
- Storage: 500 MB free disk space
- Camera: Required for scanning physical blood reports
- Internet: Broadband connection for Firebase sync

REQUIREMENT ANALYSIS

Software Requirements

- OS: Windows 10 / macOS 11 / Ubuntu 20.04 or later
- Browser: Chrome 90+, Firefox 88+, or any modern browser
- Node.js: v18+ with npm for frontend build
- Python: 3.8+ with Flask for the ML backend
- Firebase: Account for authentication and Firestore database

MODULE DESCRIPTION

Blood Evaluation Module

- Accepts blood report image or PDF upload from the user.
- Runs Tesseract.js OCR to extract text and identify parameter values.

Health Analysis Module

- Compares extracted values against standard medical reference ranges.
- Generates color-coded status: Normal, Low, or High for each parameter.

Recommendation Module

- Produces personalized diet plans, fitness tips, and lifestyle advice.
- AI chatbot answers follow-up health questions in real time.

DESIGN – ARCHITECTURAL DIAGRAM

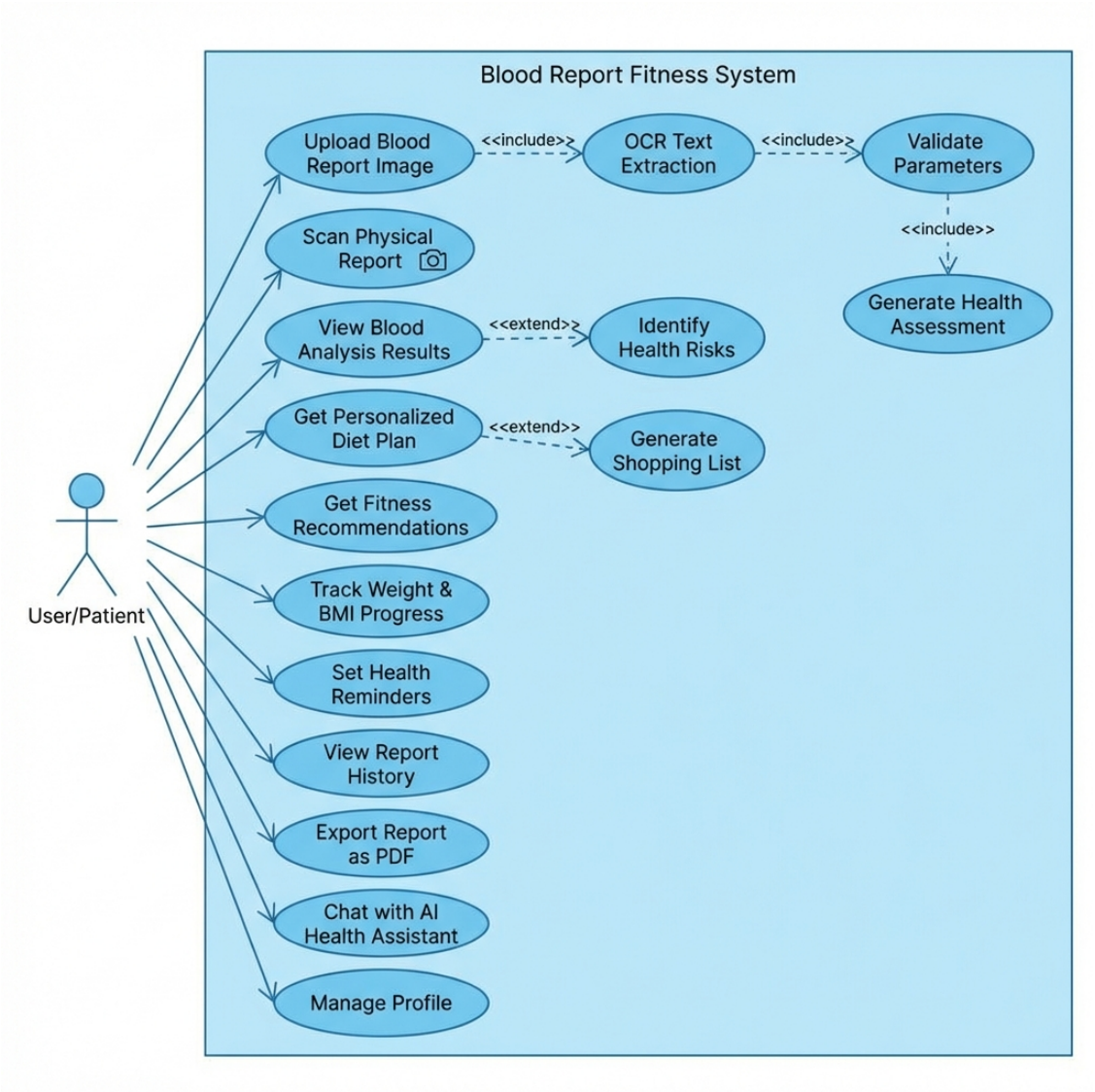
System Architecture

- User interacts with the React.js frontend via browser or Android (Capacitor).
- Tesseract.js performs client-side OCR; results stored in LocalStorage.
- Flask backend handles ML-based disease prediction via Scikit-learn.
- Firebase Firestore stores user profiles and report history in the cloud.

Layer	Component	Technology
Frontend	UI Application	React.js + Vite
OCR Engine	Text Extraction	Tesseract.js
Backend API	ML Prediction Server	Python Flask
Data Storage	Cloud Database	Firebase Firestore
Mobile	Android Wrapper	Capacitor 6.0

DESIGN – USE CASE DIAGRAM

- User uploads blood report image for automatic OCR-based analysis.
- System validates parameters and generates a health assessment report.
- User receives personalized diet, fitness, and lifestyle recommendations.
- User can track weight/BMI, export reports as PDF, and chat with AI assistant.



IMPLEMENTATION DETAILS

1. Front End

- React 18.3 SPA with Vite for fast development and production builds.
- Lucide React icons and TailwindCSS for a modern, responsive UI.
- Tesseract.js 5.0 runs OCR directly in the browser (no server needed).
- PWA support with Service Worker for offline usage and mobile install.
- State managed via React hooks; report history stored in LocalStorage.

IMPLEMENTATION DETAILS

2. Back End & Data

- Python Flask REST API exposes ML disease prediction endpoint.
- Scikit-learn Random Forest model trained on blood parameter dataset.
- Firebase Firestore stores user profiles, reports, and weight logs.
- Firebase Auth provides secure email/password login and session management.
- Capacitor 6.0 wraps React app into a native Android APK.

RESULTS AND DISCUSSION

- OCR extraction achieves over 90% accuracy on printed blood report images.
- 100+ blood parameters are analyzed and color-coded (Normal / Low / High).
- Disease risk predictions generated in under 2 seconds using the ML backend.
- Diet and fitness recommendations are context-aware and actionable.
- App tested successfully on Android devices via Capacitor APK.

TESTING

White Box Testing

- Unit tested OCR parsing logic, value extraction, and range comparison functions.
- Boundary value analysis applied to each medical parameter's min/max range.

Black Box Testing

- Functional testing of upload, analysis, and recommendation flows end-to-end.
- Invalid file types, empty reports, and edge-case values tested for robustness.

System Testing

- Integration tested frontend-to-backend ML prediction API communication.
- Performance tested OCR speed: average 5–10 seconds per blood report image.

CONCLUSION

- The system successfully automates blood report analysis using OCR and ML.
- Provides instant, personalized health insights without requiring a doctor.
- Offline-first PWA architecture ensures accessibility on mobile and desktop.
- Future scope includes cloud ML, multi-device sync, and doctor report sharing.

REFERENCES

- Kumar et al., "Automated Blood Report Analysis using CNN," IJCSE, 2021.
- Singh et al., "OCR for Medical Documents using Tesseract," IJECE, 2020.
- Patel et al., "Health Recommender Systems: A Survey," IEEE Access, 2022.
- Ali et al., "ML Disease Prediction from Blood Parameters," Elsevier, 2019.
- React.js Documentation – https://react.dev
- Tesseract.js Documentation – https://tesseract.projectnaptha.com
- Firebase Docs – https://firebase.google.com/docs